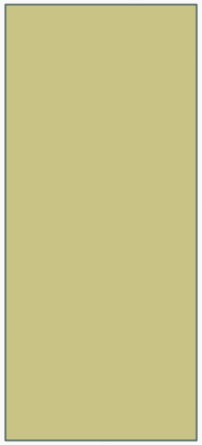
EC-200 DATA STRUCTURES

LECTURE 8



WHAT WILL BE THE OUTPUT

```
void fun( )
{
cout<< “Hello World”;
fun( );
}

void main( )
{
fun( );
}
```

WHAT WILL BE THE OUTPUT

```
void fun( )
{
fun( );
cout<< “Hello World”;
}

void main( )
{
fun( );
}
```

WHAT WILL BE THE OUTPUT

```
void fun(int i )
{
i=i-1;
cout<<i;
fun(i);
}

void main( )
{
Int j=8;
fun(j);
}
```

THINK?

What amendments in the code given below will result in printing of Hello World 10 times on console?

```
void fun( )
{
cout<< “Hello World”;
fun( );
}
void main( )
{
fun( );
}
```

RECURSION

- Recursive tasks:
  - A task that is defined in terms of itself.
  - A function that calls itself.
    - With each invocation, the problem is reduced to a smaller task (**reducing case**) until the task arrives at some **terminal case**.

RECURSION

- A recursive function has two parts:
  - the terminal/base case.
    - - a stopping condition
  - the reducing case/recursive step
    - an expression of the computation or definition in terms of itself

GENERAL ALGORITHM FOR RECURSION

```
if
   (terminal_conditio
   n)
   terminal_case
else
    reducing_case
```

```
if (!terminal_condition)
   reducing_case
```

THE FACTORIAL FUNCTION

- **n!** = n * (n-1) * (n -2) * … * 2 * 1
- **5!** = 5 * 4 * 3 * 2 * 1
- The same function can be defined recursively as fallows:
  - **0! = 1 – terminal case**
  - n! = n * (n - 1)! - the reducing case

THE FACTORIAL FUNCTION

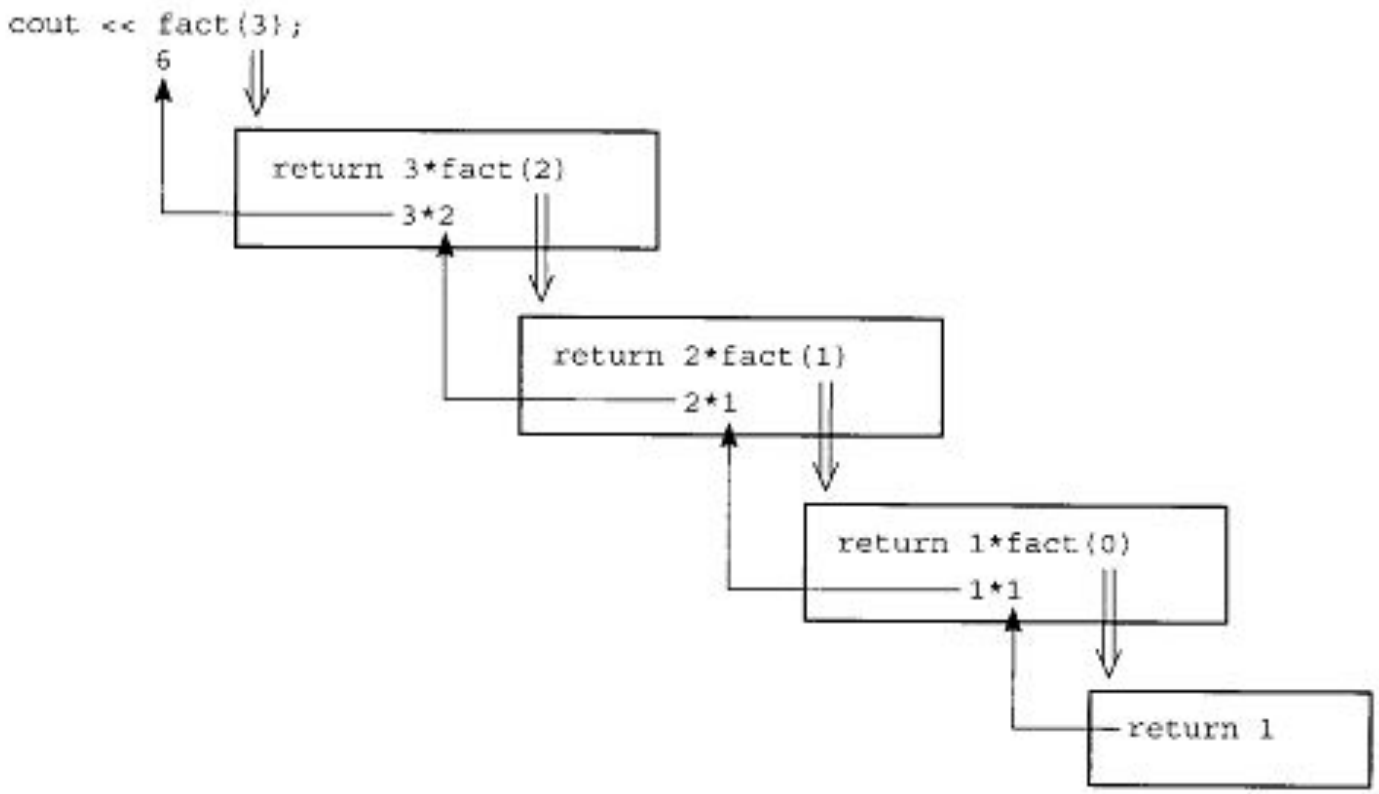
- 5! = 5 * 4!
- 4! = 4 * 3!
- 3! = 3 * 2!
- 2! = 2 * 1!
- 1! = 1! * 0!
- **0! = 1** - *Terminal Case*

Reducing Case

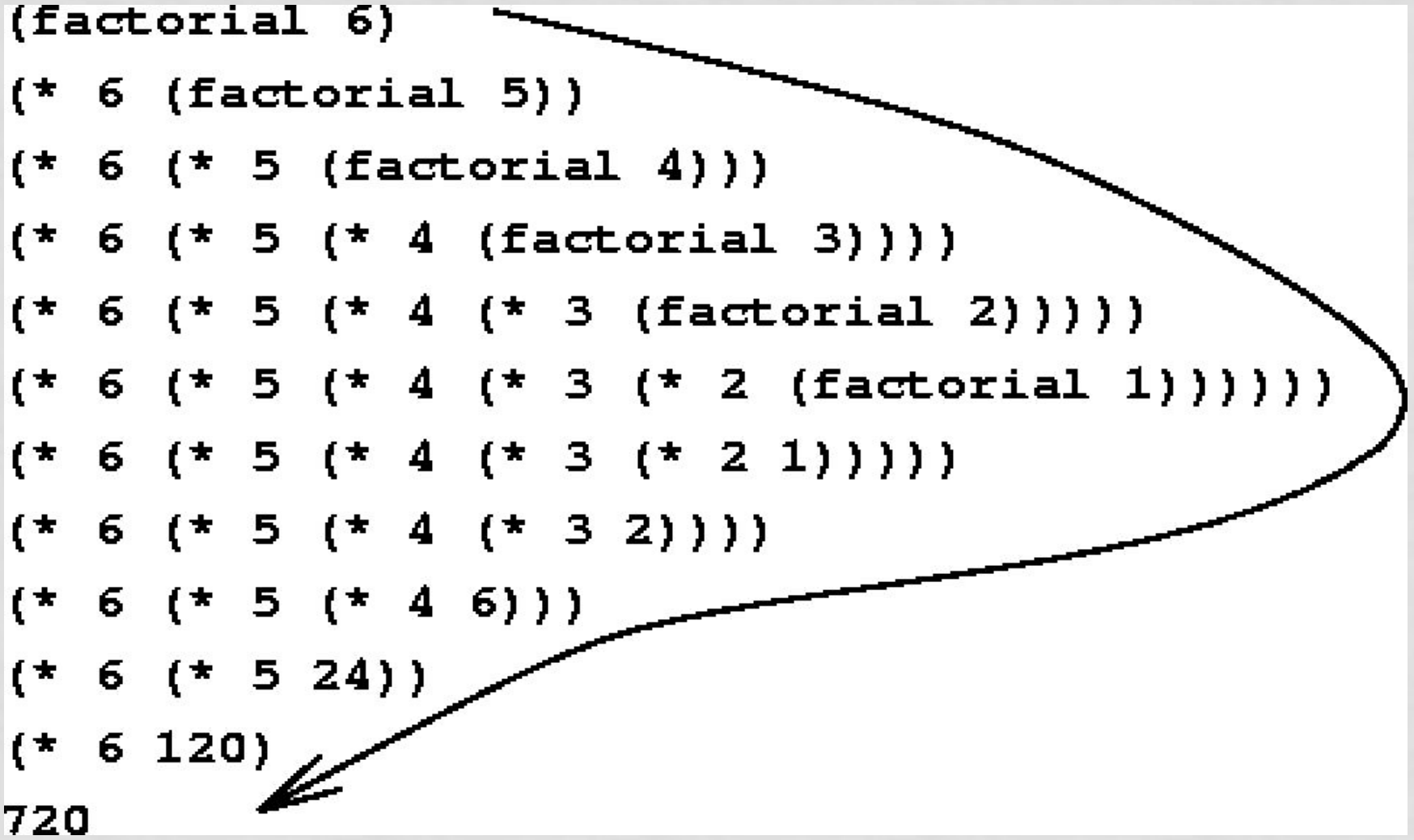
FACTORIAL RECURSIVE FUNCTION

```
int fact(int n)
// ---------------------------------------------
// Computes the factorial of a nonnegative integer.
// Precondition: n must be greater than or equal to 0.
// Postcondition: Returns the factorial of n; n is
// unchanged.
// ---------------------------------------------
{
   if (n == 0)
      return 1;
   else
      return n * fact(n-1);
}  // end fact
```

RECURSIVE CALLS IN FACTORIAL



RECURSIVE CALLS IN FACTORIAL



CLASS TASKS

- Write a recursive function in linked list ADT, traverse to display data in list
- Write a recursive function in linked list ADT search(node*ptr,int data) to search data in list
- Write a recursive function power(int n, int x) to find n^x


SAFELY ENDANGERED
SWEET JESUS, POOH!
THAT'S NOT HONEY
YOU'RE EATING
RECURSION
SAFELY ENDANGERED
SWEET JESUS, POOH!
THAT'S NOT HONEY
YOU'RE EATING
RECURSION

READING

- Chapter 2 & 5, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- https://www.youtube.com/watch?v=B0NtAFf4bvU